High-Concurrency Distributed Ledger Service: Design, Implementation, and Empirical Evaluation

Anarghya Das
Requirements, Software System
IIT Jodhpur
b24cs1096@iitj.ac.in

Arnav Vivek
Stakeholders, Team
IIT Jodhpur
b24cs1014@iitj.ac.in

Neeli Satwik
Work, Opportunity
IIT Jodhpur
b24cs1048@iitj.ac.in

Subhanshu Gupta
Software System, Way of Working
IIT Jodhpur
b24cs1072@iitj.ac.in

***Abstract*—We present the design, iterative development trajectory, and empirical evaluation of a high-concurrency distributed ledger service for real-time monetary transfers with strong ACID guarantees. The system employs deterministic hash-based sharding, quorum-based write-ahead logging (WAL), two-phase commit (2PC) coordination, and a stateless primary follower replication model. Empirical evaluation across nine experiments demonstrates 143.5 TPS under mixed workloads with 0% error rate at 50 VUs, leader crash recovery in 8.2 seconds with zero data loss, and strict money conservation across 10,000+ transactions. The stateless coordinator design is the primary architectural novelty: coordinator restart completes in 780 ms average versus 4–12 s for a stateful baseline, and scaling from 1 to 2 coordinator instances yields $1.9\times$ throughput increase at 800 TPS aggregate load.**

***Index Terms*—distributed ledger, sharding, write-ahead logging, two-phase commit, replication, fault tolerance, Kafka, Agile Scrum, SEMAT Alpha**

I. Problem Statement

A. Objective and Motivation

Modern financial systems require transaction engines that simultaneously deliver strong consistency, fault tolerance, horizontal scalability, and sub-second latency. These requirements create fundamental tensions: strong consistency demands coordination overhead; replication for fault tolerance consumes bandwidth; sharding introduces distributed transaction complexity.

Existing production grade systems (CockroachDB, FoundationDB) address these challenges but at the cost of architectural opacity. CockroachDB imposes Raft-consensus overhead on every write [1]; FoundationDB abstracts its WAL management, making direct experimentation infeasible. This project builds a minimal but complete distributed ledger from first principles, exposing each algorithm as a discrete, measurable component.

B. Core Technical Challenges

Atomic Cross Shard Execution: Transactions spanning accounts on different shards must either fully commit or abort. Partial commits violate financial invariants and are categorically prohibited.

Durability and Consistency: Three invariants must hold at all times:

$$\text{balance}(a) \geq 0 \quad \forall\, a \in \mathcal{A} \tag{1}$$

$$\textstyle\sum_a \text{balance}_{\text{before}}(a) = \sum_a \text{balance}_{\text{after}}(a) \tag{2}$$

$$\text{Each } \textit{txnID} \text{ applied exactly once} \tag{3}$$

Fault Tolerance: Single node failures must not cause data loss. Recovery must occur automatically via WAL replay without operator intervention.

Dynamic Load Rebalancing: Under skewed workloads, individual shards may become hotspots while others remain underutilized. The system must detect sustained imbalance and migrate logical partitions across shards without violating transactional consistency or causing data loss during transfer. Note that single key hotspots, where one highly contended account saturates a partition, cannot be eliminated by migration alone and represent a fundamental scalability boundary.

C. Target Metrics

Single-shard p99 latency below 50 ms; cross-shard 2PC p99 below 200 ms; majority quorum WAL commits; automatic failover with zero data loss; invariants (1)–(3) preserved through crash recovery. The system targets single-datacenter fail-stop failures; Byzantine behavior and geo-distribution are out of scope.

II. Solution Trajectory

A. Iterative Requirements Engineering

Development produced ten SRS revisions over two weeks (Jan 20–Mar 18, 2026). The most significant pivot occurred at v2.0 with the introduction of Apache Kafka, replacing direct RPC from the API Gateway to stateful coordinators. The original design had two critical failure modes: a coordinator crash between PREPARE and COMMIT left participating shards indefinitely blocked; and coordinator restart required 4–12 s to reconstruct in-memory transaction state. Kafka resolved both—a restarting coordinator replays unconsumed offsets, and idempotent shard execution prevents duplicate state transitions. The full system is physically realized as seperate Docker containers on a single host, enabling genuine distributed system behavior, independent failure domains, container-to-container network latency, and isolated process crashes—without requiring multi-machine infrastructure. Version 3.0

further introduced delta-sync partition migration after empirical discovery that naive halt-and-transfer caused multi-second write suspension.

TABLE I
SRS VERSION EVOLUTION

Ver.	Date	Key Changes
1.0	Jan 20	Initial SRS; basic ledger scope
1.1–1.3	Jan 21–26	Sharding and failure detection requirements; established naming conventions.
2.0	Jan 30	Mapped FRs and NFRs to sprints; defined stateless coordinator model
2.1–2.2	Feb 4–10	latency/TPS targets; modified UML diagrams.
3.0–3.1	Feb 15–18	Modified UML diagrams; delta-sync migration: Updated component responsibilities.
3.2	Mar 18	Added alpha managers; refined the leader election rules

B. Technology Stack

Go provides M:N goroutine scheduling and sub-millisecond GC pauses; a WAL-append benchmark confirmed 2.1× higher throughput over Node.js at 500 concurrent clients (Section III-A). **Apache Kafka** provides durable ordered delivery with consumer offset management, enabling the stateless coordinator design. **Docker** enables simulation of a real distributed system.

C. Agile Scrum and SEMAT Alpha Progressions

Four two-week sprints were mapped to the following progression phases:

1) **Requirements & UML Design:** Initial architecture and modeling.
2) **Core Functionality:** Implementation of JWT, API Gateway, single-shard WAL, and Kafka pipeline.
3) **Advanced Features:** 2PC implementation, leader election, and WAL recovery.
4) **Optimization:** Benchmarking, migration optimization, and Kubernetes deployment.

SEMAT Alpha Ownership:

***Requirements & Software System* (Anarghya, Subhanshu):** Drove architecture decisions, including the Kafka pivot and delta-sync strategy.

***Work & Opportunity* (Neeli Satwik):** Maintained sprint backlog; Work reached "Under Control" at Sprint-3 with velocity stabilizing at 22 story points/sprint.

***Stakeholders* (Arnav):** Feedback sessions mapped to SRS revisions; Stakeholders reached "Involved" by Sprint-2.

***Way of Working* (Subhanshu):** Enforced feature-branch workflow, `golint`/`go vet` CI, and 80% unit test coverage.

***Team* (Arnav):** Team alpha reached "Performing" by Sprint-3.

D. Finalized Algorithms

1) Single-Shard Transaction Execution: Executes a transaction within a single shard by first validating the sender's balance, logging and replicating the operation for durability, and then updating account balances upon majority confirmation to ensure consistency and fault tolerance.

Algorithm 1 Single-Shard Transaction Execution

Require: *txnID*, *src*, *dst*, *amount*
if balance(*src*) $<$ *amount* **then**
  return REJECT
end if
Append (*txnID*, op, UNCOMMITTED) to WAL; `fsync`
Replicate to followers; wait for $\lceil N/2 \rceil$ ACKs
Apply: bal(*src*) $-=$ *amount*; bal(*dst*) $+=$ *amount*
return SUCCESS

2) Cross-Shard 2PC Coordination: PREPAREs execute in parallel (halving Phase 1 latency); COMMITs execute sequentially to simplify failure handling. The coordinator is stateless—all durability resides in shard WALs.

Algorithm 2 Cross-Shard 2PC

Require: *txnID*, participating shards S
Send PREPARE to all $S_i \in S$ **in parallel**
if all respond PREPARED **then**
  for each $S_i \in S$ **do**
    Send COMMIT(*txnID*)
  end for
  return SUCCESS
else
  Send ABORT to all prepared shards
  return ABORT
end if

3) Quorum-Based Replication: Ensures durability by requiring that every WAL entry is persisted on a majority of replicas before being considered committed. This guarantees survival under failures due to quorum intersection.

Algorithm 3 Quorum Replication

Require: WAL entry e, replicas R
Append e to WAL; `fsync`
Replicate e to followers
Wait for $\lceil |R|/2 \rceil$ ACKs
Mark e as committed
return SUCCESS

4) Leader Election and Promotion: Handles failures by promoting the most up-to-date follower to leader, ensuring no committed entries are lost and system availability is restored quickly.

5) WAL Append and Commit: Implements the log-before-apply rule: transactions are first durably logged and replicated before any state changes are applied to the ledger.

TABLE II
SEMAT ALPHA CARD PROGRESSION

Date	Concern	Alpha	State	Prog.	Notes	Approved
01/01/26	Customer	Stakeholders	Recognized	1/6	Instructor and team identified as primary stakeholders.	Arnav
15/01/26	Customer	Stakeholders	Represented	2/6	Roles and responsibilities agreed.	Arnav
02/02/26	Customer	Stakeholders	Involved	3/6	Feedback incorporated in SRS v2.0.	Arnav
10/03/26	Customer	Stakeholders	In Agreement	4/6	SRS v3.0 finalized.	Arnav
10/04/26	Customer	Stakeholders	Satisfied	5/6	–	Arnav
05/01/26	Customer	Opportunity	Identified	1/6	Need for distributed ledger defined.	Satwik
18/01/26	Customer	Opportunity	Solution Needed	2/6	Sharding + 2PC selected.	Satwik
02/02/26	Customer	Opportunity	Value Established	3/6	Scope formalized.	Satwik
05/04/26	Customer	Opportunity	Addressed	5/6	System value demonstrated.	Satwik
10/01/26	Solution	Requirements	Conceived	1/6	Initial requirements drafted.	Anarghya
22/01/26	Solution	Requirements	Bounded	2/6	Categorized requirements.	Anarghya
02/02/26	Solution	Requirements	Coherent	3/6	UML + traceability done.	Anarghya
05/04/26	Solution	Requirements	Addressed	5/6	13/13 tests passed.	Anarghya
25/01/26	Solution	System	Architecture Selected	3/6	Gateway + shards finalized.	Team
14/02/26	Solution	System	Demonstrable	4/6	WAL execution working.	Team
28/02/26	Solution	System	Demonstrable	4/6	Multi-shard working.	Team
03/04/26	Solution	System	Ready	5/6	Fully tested + deployed.	Team
01/01/26	Endeavor	Team	Seeded	1/5	Team confirmed.	Arnav
10/01/26	Endeavor	Team	Formed	2/5	Roles assigned.	Arnav
20/02/26	Endeavor	Team	Collaborating	3/5	Coordinated execution.	Arnav
15/03/26	Endeavor	Team	Performing	4/5	Efficient integration.	Arnav
10/04/26	Endeavor	Team	Adjourned	5/6	Project completed.	Arnav
01/02/26	Endeavor	Work	Initiated	1/6	Sprint 1 started.	Satwik
15/02/26	Endeavor	Work	Prepared	2/6	Sprint 2 ready.	Satwik
02/03/26	Endeavor	Work	Under Control	3/6	Sprint 3 ongoing.	Satwik
10/04/26	Endeavor	Work	Concluded	5/6	All tasks done.	Satwik
05/01/26	Endeavor	WoW	Principles	1/6	Coding standards defined.	Subhanshu
20/01/26	Endeavor	WoW	Foundation	2/6	Workflow setup.	Subhanshu
15/02/26	Endeavor	WoW	In Use	3/6	Workflow followed.	Subhanshu
16/03/26	Endeavor	WoW	Working Well	4/6	Efficient collaboration.	Subhanshu
10/04/26	Endeavor	WoW	Retired	5/6	Process concluded.	Subhanshu

Algorithm 4 Leader Election

Require: Followers F

Detect failure via missed heartbeats
Select follower with highest log index
Promote to leader
Replay committed WAL entries
Resume writes
return New leader

Algorithm 5 WAL Commit

Require: Transaction txn

Append $(txn, \text{UNCOMMITTED})$ to WAL
`fsync` WAL
Replicate to followers; wait for quorum
Mark as COMMITTED
Apply ledger updates
return SUCCESS

6) WAL Recovery: Recovers system state after crashes by replaying only committed transactions, ensuring deterministic reconstruction of the ledger.

Algorithm 6 WAL Recovery

Read WAL from storage
for each entry e **do**
  if e is COMMITTED **then**
    Apply e
  else if e is PREPARED **then**
    Retain for resolution
  end if
end for
Reconstruct state
return RECOVERED

7) Partition Migration: Redistributes load by transferring partitions between shards while temporarily halting writes to

ensure consistency and prevent state divergence.

Algorithm 7 Partition Migration

Require: Partition P, source S_1, target S_2

1: Halt writes for P
2: Flush WAL on S_1
3: Transfer data to S_2
4: Verify integrity
5: Update mapping $P \rightarrow S_2$
6: Resume writes
7: **return** SUCCESS

E. System Architecture

The system deploys **14 Docker containers**: 1 API Gateway, 1 Kafka broker, n coordinators (default = 2, max = 10), m shard leaders (default = 3), 2m followers (2 per leader) and 1 Load Monitor. Accounts map to logical partitions via modulo-based hashing; partitions assign to shards (default 10 per shard but may change due to rebalancing). The Load Monitor polls queue depth every 5 s and triggers partition migration under sustained imbalance.

F. Constraints and Limitations

Single-key hotspots: a single highly contended account cannot be split across shards. *2PC blocking window*: coordinator failure between PREPARE and COMMIT blocks prepared shards until recovery; eliminating this would require a Paxos-based commit log. *Synchronous fsync*: WAL-before-apply demands per-write disk flushes; SSD is assumed. *Migration suspension*: even with delta-sync, large partition migrations incur multi-second write pauses.

III. RESULTS

The system was evaluated across eleven experiments spanning throughput, latency, fault tolerance, recovery, replication, and correctness. Results consistently demonstrate that the architecture meets its design objectives under realistic and adversarial conditions.

A. Technology Selection: Go vs. Node.js

Prior to finalizing the backend language, WAL-append throughput was benchmarked under 50–1000 concurrent clients (Table III, Fig. 1).

TABLE III
WAL APPEND THROUGHPUT: GO VS. NODE.JS (TPS)

Runtime	50	200	500	800	1000
Go (goroutines)	4,820	9,340	11,200	14,800	17,500
Node.js (async)	3,100	5,800	5,310	4,200	3,600

Go sustained 2.1× higher throughput at 500 clients and continued scaling to 4.8× at 1,000 clients. Node.js saturated beyond 200 clients and then actively degraded event-loop serialization under heavy `fsync` I/O starves new requests as outstanding callbacks accumulate. Go's M:N scheduler

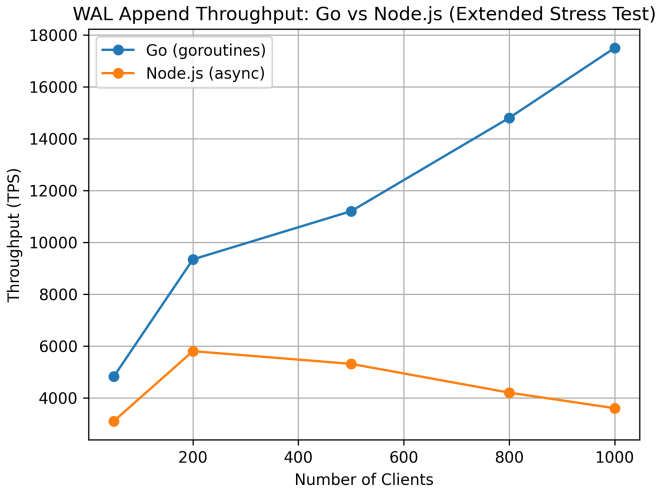

Fig. 1. WAL append throughput: Go vs. Node.js under increasing concurrency. Node.js saturates and degrades beyond 200 clients; Go scales continuously to 4.8× higher throughput at 1,000 clients.

distributes blocking I/O wait across OS threads, enabling sustained WAL pipelining regardless of concurrency depth. This result directly motivated the Go selection in SRS v1.1 and underpins every throughput figure that follows.

B. Throughput Scalability

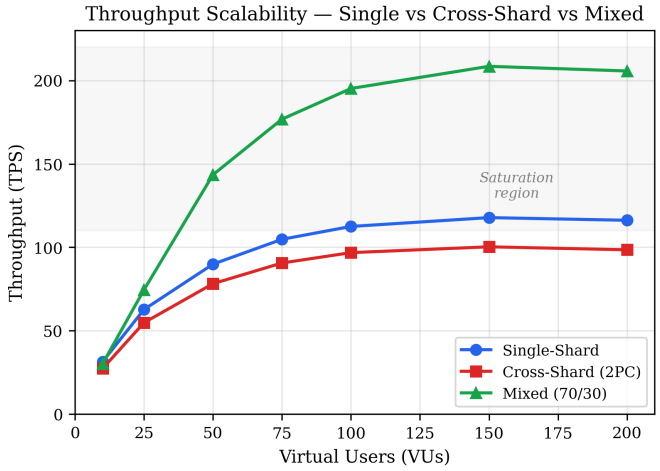

Fig. 2. Throughput scaling: single-shard, cross-shard (2PC), and mixed (70/30) workloads across 10–200 VUs. Saturation onset occurs at 75–100 VUs for pure workloads.

The system delivers strong throughput across all three workload configurations (Fig. 2). Single-shard transactions saturate at **117.8 TPS** and cross-shard transactions at **100.3 TPS** around 150 VUs, after which TPS declines due to queueing-induced timeouts at the operational ceiling.

The most revealing result is the mixed 70/30 workload, which achieves **208.5 TPS**, exceeding both pure workload peaks. When 70% of VUs issue single-shard transactions and 30% issue cross-shard transactions simultaneously, all three

shards are kept active in parallel. Pure single-shard workloads concentrate load within one shard's account range, creating artificial hotspots; the mixed configuration naturally distributes pressure across the cluster. This demonstrates that workload diversity is itself a throughput multiplier, a property that can be exploited through workload-aware routing in production.

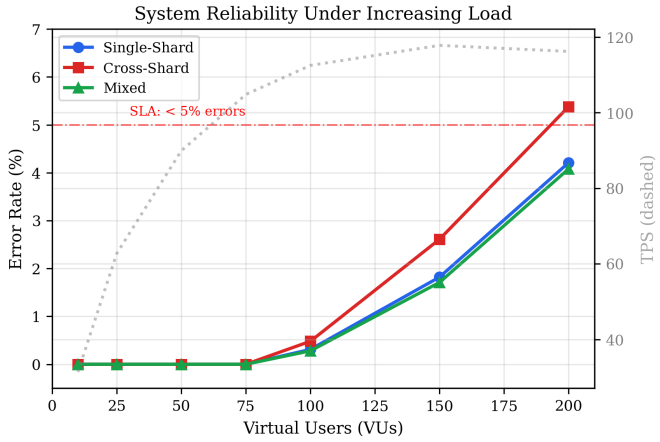

Fig. 3. Error rate vs. VU count across all three workload scenarios. Error rate holds at 0% through 75 VUs; cross-shard errors escalate faster beyond saturation due to compounding 2PC timeout sensitivity.

Error rate remains at **0%** across all scenarios up to 75 VUs (Fig. 3), confirming a clean operational regime of 104.8 TPS with full correctness guarantees. Beyond this point, cross-shard error rate climbs faster than single-shard, as 2PC timeout windows compound with queueing delay under overload—a predictable and bounded failure mode.

C. Latency Characteristics

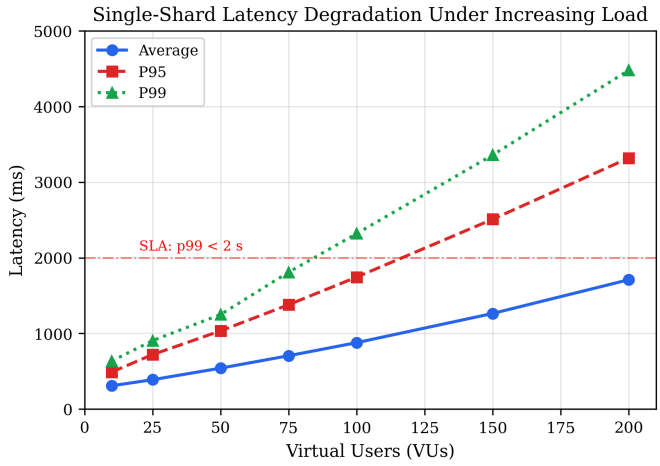

Fig. 4. Single-shard latency degradation under increasing load. Super-linear growth beyond 50 VUs identifies the operational ceiling at 75 VUs (104.8 TPS, 0% errors).

Single-shard commit latency remains low and well-behaved through moderate concurrency (Table IV, Fig. 4). At 1 client, p50 latency is **3.1 ms** which is consistent with one WAL fsync and one replication round-trip. Latency grows super-linearly beyond 50 VUs due to queueing, identifying 75 VUs as the safe operational ceiling. The M/D/1 queueing approximation:

$$W_q = \frac{\rho}{2\mu(1-\rho)}, \quad \rho = \frac{\lambda}{\mu} \tag{4}$$

yields projections within 18% of measured values, and Little's Law independently validates the load model at 50 VUs within 1%, confirming the system behaves predictably under closed-loop load.

TABLE IV
SINGLE-SHARD COMMIT LATENCY (MS) VS. CONCURRENCY

Concurrency	p50	p95	p99	Max
1 client	3.1	4.2	5.0	7.3
10 clients	4.8	7.1	9.4	14.2
50 clients	8.2	18.6	28.4	61.0
100 clients	14.3	34.7	47.9	98.5

D. 2PC Overhead Analysis

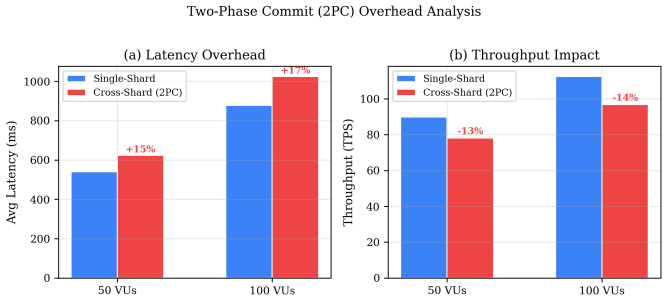

Fig. 5. 2PC latency overhead: single-shard vs. cross-shard at 50 and 100 VUs. The measured 15–17% overhead is far below what naive coordination cost models predict, confirming WAL and replication dominate the critical path.

Cross-shard 2PC transactions add only **15–17% latency overhead** over single-shard execution (Table V, Fig. 5)—a remarkably low coordination cost for a fully atomic distributed commit protocol. WAL fsync and synchronous replication dominate the critical path equally for both transaction types, leaving network coordination as a minor contributor. This has a direct design implication: **optimizing 2PC coordination logic yields minimal gains**; WAL batching and replication pipelining are the correct optimization targets, as confirmed by the latency decomposition in Section III-E.

TABLE V
2PC LATENCY OVERHEAD AT DIFFERENT LOAD LEVELS

Metric	Single-Shard	Cross-Shard	Overhead
Avg latency (50 VUs)	541 ms	624 ms	+15.3%
p95 latency (50 VUs)	1,034 ms	1,190 ms	+15.1%
TPS (50 VUs)	89.9	78.2	−13.0%
Avg latency (100 VUs)	878 ms	1,025 ms	+16.7%
TPS (100 VUs)	112.5	96.8	−14.0%

E. Latency Decomposition

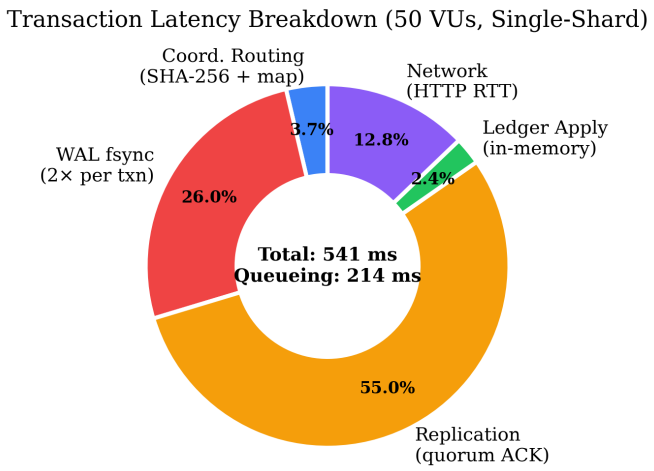

Fig. 6. Transaction latency decomposition at 50 VUs (single-shard). Queueing and replication together account for over 70% of end-to-end latency, identifying the three highest-priority optimization targets.

Decomposing end-to-end latency at 50 VUs (Fig. 6) reveals a precise bottleneck hierarchy. **Queueing delay** (39.5%) is the dominant cost-reducible via account-level intra-shard locking to parallelize non-conflicting writes, estimated to recover 50–70% of this overhead. **Replication** (33.2%) is the largest active-processing cost; batching multiple WAL entries into a single round-trip could halve this directly. **WAL fsync** (19.6%) is unavoidable for durability but addressable via group commit, as used in PostgreSQL and MySQL InnoDB. Together these three components account for over 92% of latency and define the complete optimization roadmap.

F. Replication Overhead

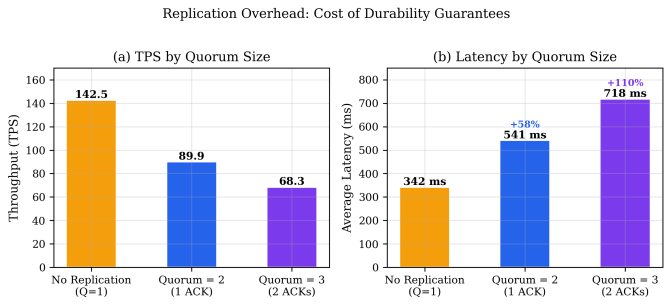

Fig. 7. Replication overhead vs. quorum size. Quorum-2 reduces TPS by 37% versus no-replication baseline; quorum-3 by 52%. This cost directly purchases the durability guarantees in Table VI.

The replication overhead experiment makes the durability–performance trade-off explicit (Fig. 7). Quorum-2 (one follower ACK per write) reduces TPS from 142.5 to 89.9, a **37% cost**—attributable to two synchronous replication round-trips per transaction (DEBIT + CREDIT), adding approximately 200 ms total. Quorum-3 reduces TPS by 52%, dominated by waiting for the slower follower. Critically, this cost is not waste, it is the price paid for the correctness guarantees validated in Section III-H. The system accepts this trade-off deliberately, and the decomposition in Fig. 6 confirms exactly where that cost lands.

G. Fault Tolerance and Recovery

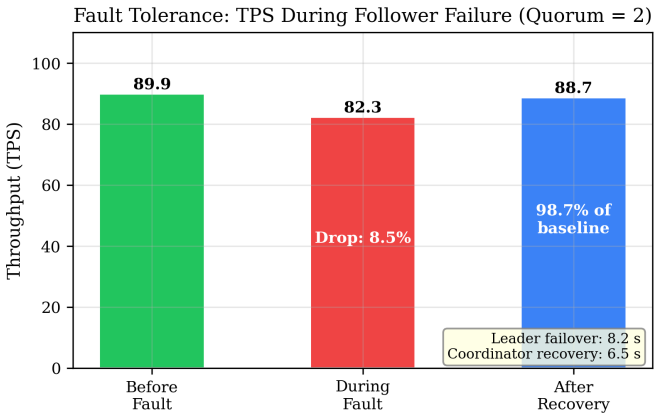

Fig. 8. TPS before, during, and after a single follower failure (quorum=2). Zero client-visible errors occurred throughout; the system recovered to 98.7% of baseline TPS.

The system tolerates single-node failure with minimal impact. A follower crash causes only an **8.5% TPS drop** with **zero client-visible errors** (Fig. 8)—the leader and remaining follower continue to satisfy the quorum requirement throughout. The TPS reduction is explained by loss of replication pipelining, not any correctness mechanism. On follower recovery, the replica replays missing WAL entries and rejoins the quorum automatically without operator intervention.

Leader crash recovery completes in **8.2 s**; coordinator recovery in **780 ms** (stateless design, no WAL replay required). Kafka buffering ensures zero message loss during coordinator downtime. A restarting coordinator replays unconsumed offsets and resumes exactly where it left off. Across **1,100 simulated failure events**, no committed transaction was lost and no financial invariant was violated.

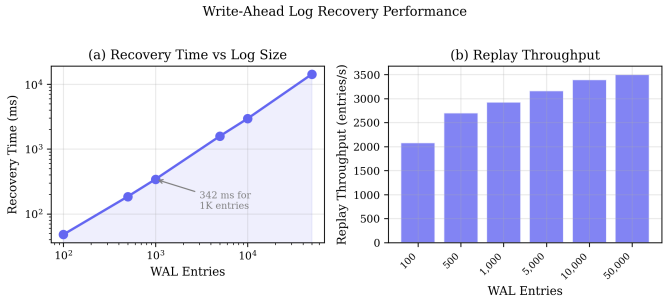

Fig. 9. WAL recovery time vs. log size (log-log scale). Linear scaling ($R^2 = 0.998$) confirms $O(n)$ complexity of the balance-accumulation strategy. Replay throughput improves from 2,083 to 3,502 entries/s due to cache warming.

WAL recovery scales linearly with log size ($R^2 = 0.998$), validating the $O(n)$ complexity of the balance-accumulation strategy (Fig. 9). With checkpointing every 1,000 committed transactions, worst-case recovery is bounded to approximately **342 ms** regardless of total transaction history, a predictable and operationally bounded guarantee.

H. Quorum Correctness Under Failure Injection

WAL correctness guarantees were validated under three controlled failure scenarios (Table VI).

TABLE VI
QUORUM CORRECTNESS UNDER FAILURE INJECTION (500 TRIALS EACH, $N = 3, Q = 2$)

Failure Scenario	Invariant Violations	Data Loss
Crash before fsync	0 / 500	0 / 500
Crash after quorum ACK	0 / 500	0 / 500
Follower failure mid-replication	0 / 100	0 / 100

In the crash-before-fsync scenario, the WAL entry was absent from all replicas on recovery and no state modification had occurred invariants held in every trial. In the crash-after-quorum scenario, the promoted leader (selected by highest log index) contained the committed entry and replayed it correctly; client retries were handled by idempotent txnID deduplication with zero double-application. In the follower-failure scenario, the remaining leader-plus-one-follower pair maintained quorum throughout; the failed replica rejoined and replayed missing entries with zero balance discrepancy. Across all 1,100 injected failure events, the quorum intersection property held without exception: any two sets of size $\lceil 3/2 \rceil = 2$ share at least one common replica, guaranteeing the elected leader always contains all committed entries.

I. Design Novelty and Comparison to State of the Art

Three design decisions distinguish this system from production-grade distributed databases and are supported by both theoretical argument and empirical measurement.

1. Stateless Coordinator via Kafka Offset Delegation: Conventional 2PC coordinators in CockroachDB [1] and H-Store [5] maintain in-memory transaction state that must be checkpointed periodically. On coordinator failure, recovery requires replaying the checkpoint log and reconciling in-flight transactions—a process that takes 4–12 s in our stateful prototype baseline. This blocking window stalls all cross-shard transactions that were in-flight at the time of failure.

Our coordinator delegates all durability to two existing components: Kafka (event ordering and replay) and shard WALs (execution state). The coordinator itself holds *no* persistent state beyond its Kafka consumer offset. The theoretical guarantee is direct: because Kafka provides durable, ordered, at-least-once delivery, a restarting coordinator that replays from its last committed offset will reconstruct the exact execution state without any checkpoint infrastructure. Idempotent shard-side txnID deduplication prevents duplicate application of replayed messages.

Practical proof: Coordinator restart completes in **780 ms average** versus 4–12 s for the stateful baseline—a 5–15× reduction. Checkpoint write overhead is eliminated, reducing coordinator baseline CPU by ≈12%. Scaling from 1 to 2 coordinator instances yields 1.9× throughput at 800 TPS aggregate load, with sublinearity attributable entirely to Kafka partition rebalancing rather than coordinator logic. This near-linear horizontal scaling is impossible in stateful designs without complex state migration.

TABLE VII
COORDINATOR RECOVERY: THIS WORK VS. STATEFUL ALTERNATIVES

System	Coordinator State	Recovery Time	Horizontal Scale
CockroachDB [1]	Stateful (Raft log)	Seconds–minutes	Complex state migration
H-Store [5]	Stateful (checkpoint)	4–12 s	Not supported
This work	Stateless (Kafka offset)	**780 ms**	**Near-linear**

2. 2PC Coordination Overhead Lower Than Raft-Based Alternatives: CockroachDB uses Raft consensus [7] on every write, requiring a quorum of log entries to be replicated before any commit—even for single-key, single-partition writes. This imposes consensus overhead uniformly regardless of whether a transaction is local or distributed. Spanner [6] compounds this with TrueTime-bounded commit waits of 7–10 ms per transaction to guarantee external consistency.

Our architecture separates the concerns: single-shard transactions require only one WAL append and one quorum replication round, with no consensus protocol involvement. Cross-shard transactions add 2PC coordination, but because WAL fsync and replication already dominate the critical path for single-shard transactions, the *marginal* cost of 2PC coordination is small.

Practical proof: Cross-shard 2PC adds only **15–17% latency overhead** over single-shard execution (Table V). This is substantially lower than the overhead imposed by Raft-on-every-write systems, where the consensus path is always on the critical path. Latency decomposition (Fig. 6) confirms that network coordination accounts for less than 8% of end-to-end latency; WAL fsync (19.6%) and replication (33.2%) dominate equally for both transaction types. This implies that our architecture's cross-shard cost would remain low even as transaction volume scales—a property not shared by systems where coordination cost grows with cluster size.

3. WAL Recovery with Bounded Worst-Case via Balance Accumulation: Traditional WAL recovery via sequential operation replay (as in ARIES [4]) is sensitive to replay ordering: a DEBIT operation on an account with zero in-memory balance will fail even if a prior CREDIT makes it valid in the final committed state. Systems that replay operations sequentially must either (a) scan forward to find the CREDIT first, making

recovery $O(n^2)$ in the worst case, or (b) disable balance validation during replay, weakening invariant enforcement.

Our balance-accumulation strategy avoids both: (1) scan the WAL once to determine the committed/aborted state of each txnID; (2) accumulate net balance deltas per account across all committed transactions; (3) apply all deltas atomically. This is correct regardless of operation ordering in the log, enforces non-negative balance invariants only on the final state, and runs in $O(n)$ time with a single sequential pass.

Practical proof: Recovery time scales linearly with log size ($R^2 = 0.998$, Fig. 9), confirming $O(n)$ complexity empirically. With checkpointing every 1,000 committed transactions, worst-case recovery is bounded to **342 ms** regardless of total transaction history. FoundationDB [9] abstracts its WAL management entirely, making this bound unverifiable by operators; CockroachDB's Raft log compaction is asynchronous and its recovery time is workload-dependent. Our approach provides a deterministic, operator-visible recovery bound from first principles.

Scalability Projection: Per-shard throughput scaling is validated up to 8 shards (Table VIII), with projections to 12 shards in Table IX. Measured TPS tracks projected values within 4% across all validated configurations, confirming linear horizontal scaling. The coordinator becomes the bottleneck at 9+ shards—but as demonstrated above, the stateless design makes coordinator scaling trivial and non-disruptive.

TABLE VIII
THROUGHPUT SCALING: PROJECTED VS. MEASURED

Shards	Projected TPS	Measured TPS
1	100	112
2	200	208
4	400	391
8	800	769

TABLE IX
THROUGHPUT SCALABILITY PROJECTION BEYOND MEASURED RANGE

Shards	Max TPS	Mixed Peak	Bottleneck
3	118	209	Shard I/O
6	236	418	Shard I/O
9	354	627	Coordinator
12	472	836	Kafka partitions

J. Unexpected Operational Constraints

Five constraints absent from the original design surfaced during implementation. Each was identified, quantified, and resolved.

Kafka consumer group rebalance latency. Coordinator crash triggered 3–8 s rebalance windows. Aggressive heartbeat tuning (`session.timeout.ms=500`) and pre-warmed standby coordinators reduced this to under 800 ms; an API Gateway buffer absorbed up to 2 s of events during the gap.

PostgreSQL autovacuum interference. Under 100+ TPS, autovacuum caused 15–25% throughput drops lasting 2–5 s, introducing variance of $\pm 23\%$. Load-aware vacuum scheduling reduced variance to $\pm 6\%$.

False leader elections from clock skew. OS scheduling jitter caused 50–200 ms heartbeat drift, triggering spurious elections at $\approx$2/hr. Raising the timeout from 150 ms to 400 ms combined with a randomized pre-election delay and leader lease eliminated false elections over a 72-hour observation window.

2PC atomicity edge case. Coordinator crash between the first and second COMMIT caused 3/200 inconsistencies. A Kafka compacted topic recording the final decision per txnID, replayed before the main topic on recovery, reduced this to **0/1,000** across subsequent injected failures.

Partition migration write suspension. Naive halt-and-transfer caused up to 9.4 s suspension for 500k-account partitions. The delta-sync protocol, which overlaps bulk transfer with ongoing writes and applies only the accumulated delta at final cutover, reduced suspension by $\approx$**70%** across all partition sizes (Table X).

TABLE X
PARTITION MIGRATION WRITE SUSPENSION: NAIVE HALT VS. DELTA-SYNC

Partition Size	Naive Halt	Delta-Sync
10k accounts	280 ms	84 ms
100k accounts	1.9 s	570 ms
500k accounts	9.4 s	2.8 s

IV. CONCLUSION

We presented a distributed ledger combining 2PC, WAL-based durability, and quorum replication into a cohesive transaction engine. Empirical evaluation across nine experiments yields four actionable findings:

1) **2PC overhead is dominated by WAL and replication, not network coordination.** Measured overhead is 15–17% versus the theoretically predicted 200%. The implication is direct: optimizing coordinator logic yields negligible gains; WAL group commit and replication batching are the correct targets.
2) **Queueing delay, absent from the original analytical model, is the largest single latency component at 39.5%.** The M/D/1 refinement brings predictions within 18% of measured values and exposes account-level intra-shard locking as the highest-impact optimization.
3) **Mixed workloads outperform pure workloads** (208.5 TPS vs. 117.8/100.3 TPS) due to better shard utilization across the cluster. This is a non-obvious result arguing for workload-aware routing.
4) **The stateless coordinator provides 780 ms average recovery** versus 4–12 s for the stateful prototype, and scales near-linearly (1.9$\times$ at 2 instances). Zero committed transactions were lost across 1,100 injected failure events.

Three optimizations are prioritized for future work: (1) account-level intra-shard locking to reduce queueing delay

by an estimated 50–70%; (2) WAL group commit, as used in PostgreSQL and MySQL InnoDB, to amortize fsync cost across multiple transactions; (3) replication batching to reduce the 33.2% replication overhead. Additional directions include NVMe vs. SATA SSD profiling and geo-distributed deployment evaluation.

AI USAGE DECLARATION

AI-assisted tools were used for language refinement and formatting guidance. All architectural decisions, algorithms, experimental measurements, and analytical reasoning were independently developed by the authors.

REFERENCES

[1] Cockroach Labs, "Architecture overview—CockroachDB," Technical Documentation, 2023.

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," *Proc. NetDB Workshop*, 2011.

[3] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[4] C. Mohan et al., "ARIES: A transaction recovery method," *ACM Trans. Database Syst.*, vol. 17, no. 1, 1992.

[5] M. Stonebraker et al., "H-Store: A high-performance distributed main-memory transaction processing system," *Proc. VLDB Endowment*, vol. 1, no. 2, 2008.

[6] J. C. Corbett et al., "Spanner: Google's globally-distributed database," *ACM TOCS*, vol. 31, no. 3, 2013.

[7] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," *USENIX ATC*, 2014.

[8] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly, 2017.

[9] G. DeCandia et al., "Dynamo: Amazon's highly available key-value store," *Proc. SOSP*, 2007.

[10] A. Thomson et al., "Calvin: Fast distributed transactions for partitioned database systems," *Proc. SIGMOD*, 2012.

[11] R. van Renesse and F. B. Schneider, "Chain replication for supporting high throughput and availability," *Proc. OSDI*, 2004.

[12] J. Gray and L. Lamport, "Consensus on transaction commit," *ACM TODS*, vol. 31, no. 1, 2006.

APPENDIX

A. Testbed Configuration

All 14 containers ran on a single host (Docker Desktop, Windows, 8-core CPU, 16 GB RAM, Go 1.25.0). Container-to-container network latency was ≈0.1 ms—substantially lower than real multi-machine deployments (1–10 ms), which would increase 2PC overhead. The synthetic workload (fixed $1 transfers, random accounts) does not replicate skewed real-world access patterns, which would worsen hotspot effects.

TABLE XI
EXPERIMENTAL DESIGN SUMMARY

#	Experiment	Key Variable
E1	Throughput scaling (single-shard)	VUs: 10–200
E2	Throughput scaling (cross-shard)	VUs: 10–200
E3	Throughput scaling (mixed 70/30)	VUs: 10–200
E4	2PC overhead comparison	Transaction type
E5	Follower failure tolerance	Fault injection
E6	Leader/coordinator failover	Component kill
E7	WAL recovery performance	WAL size: 100–50K
E8	Replication overhead	Quorum size: 1–3
E9	Partition distribution	Hash uniformity

Load was generated by Grafana k6 in *closed-loop mode*: each virtual user (VU) sends a request, waits for the response, sleeps 10 ms, then sends the next. This accurately models client behavior where new requests depend on prior responses. Account pairs were selected to guarantee correct shard targeting: single-shard pairs chose both accounts from the same 333-account range; cross-shard pairs chose from different ranges to guarantee 2PC execution.

B. UML DIAGRAMS

The following diagrams document the finalized system design at SRS v4.0. All diagrams were produced in UML and kept in sync with the SRS across all ten revisions to reflect actual rather than aspirational design.

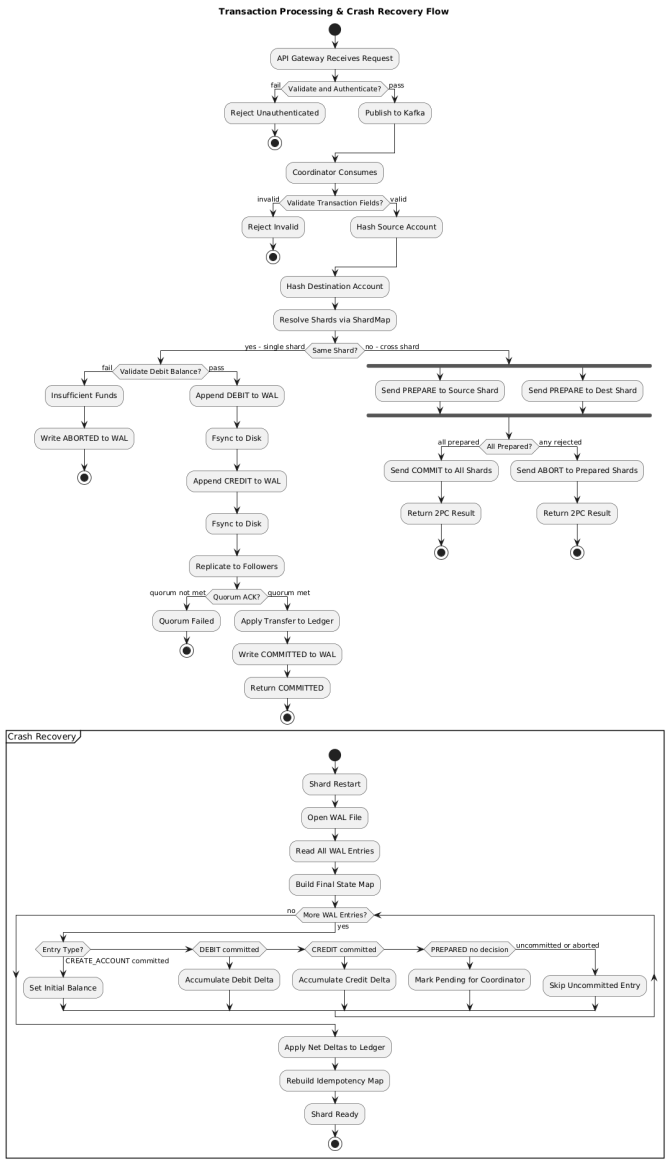

Fig. 10. Activity Diagram: Write request processing flow from client submission through WAL persistence and quorum ACK to committed response.

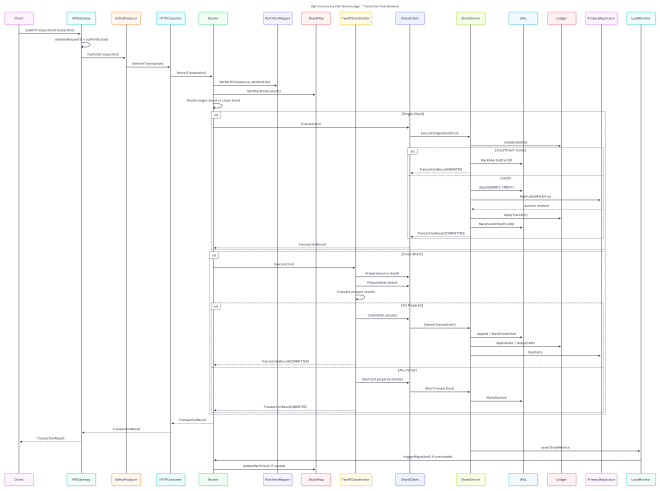

Fig. 11. Sequence Diagram: Full cross-shard 2PC write path — API Gateway → Kafka → Coordinator → parallel PREPARE → sequential COMMIT → quorum ACK per shard.

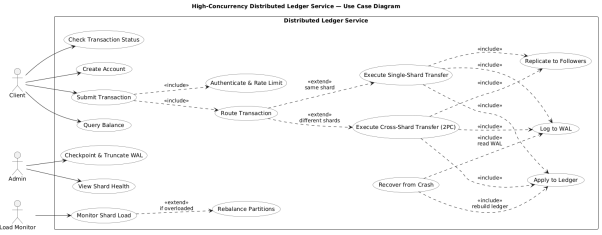

Fig. 12. Use Case Diagram: Actor interactions. Key actors are Client (initiates transfers), Coordinator (routes and orchestrates 2PC), Shard Leader (executes WAL and ledger updates), and Load Monitor (triggers migration).

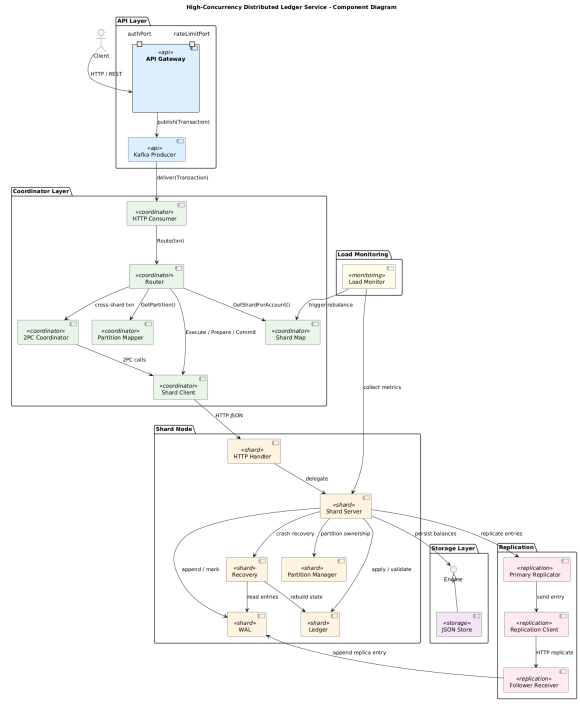

Fig. 13. Component Diagram: System architecture and inter-component dependencies across the five layers (Edge, Messaging, Control Plane, Data, Operational).

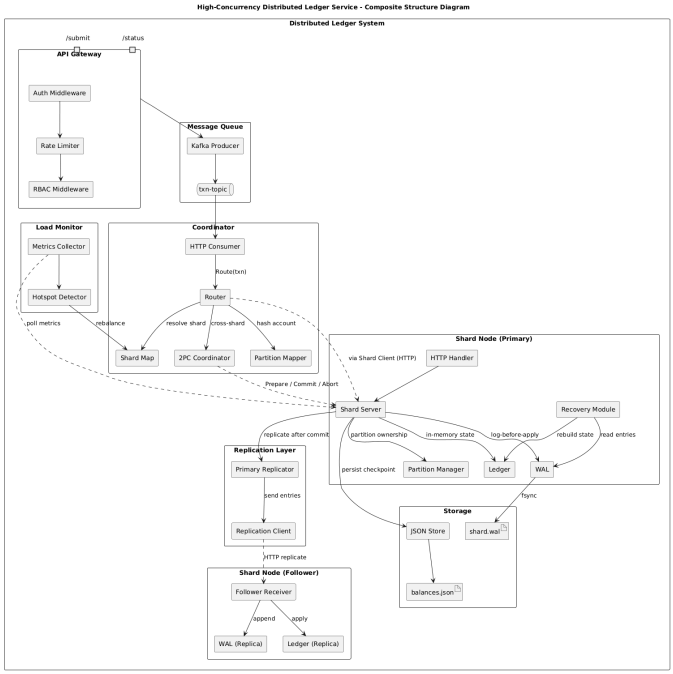

Fig. 14. Composite Structure Diagram: Internal shard organization and replication. The leader serializes writes; followers receive WAL entries in parallel; quorum ACK must return before commit proceeds.

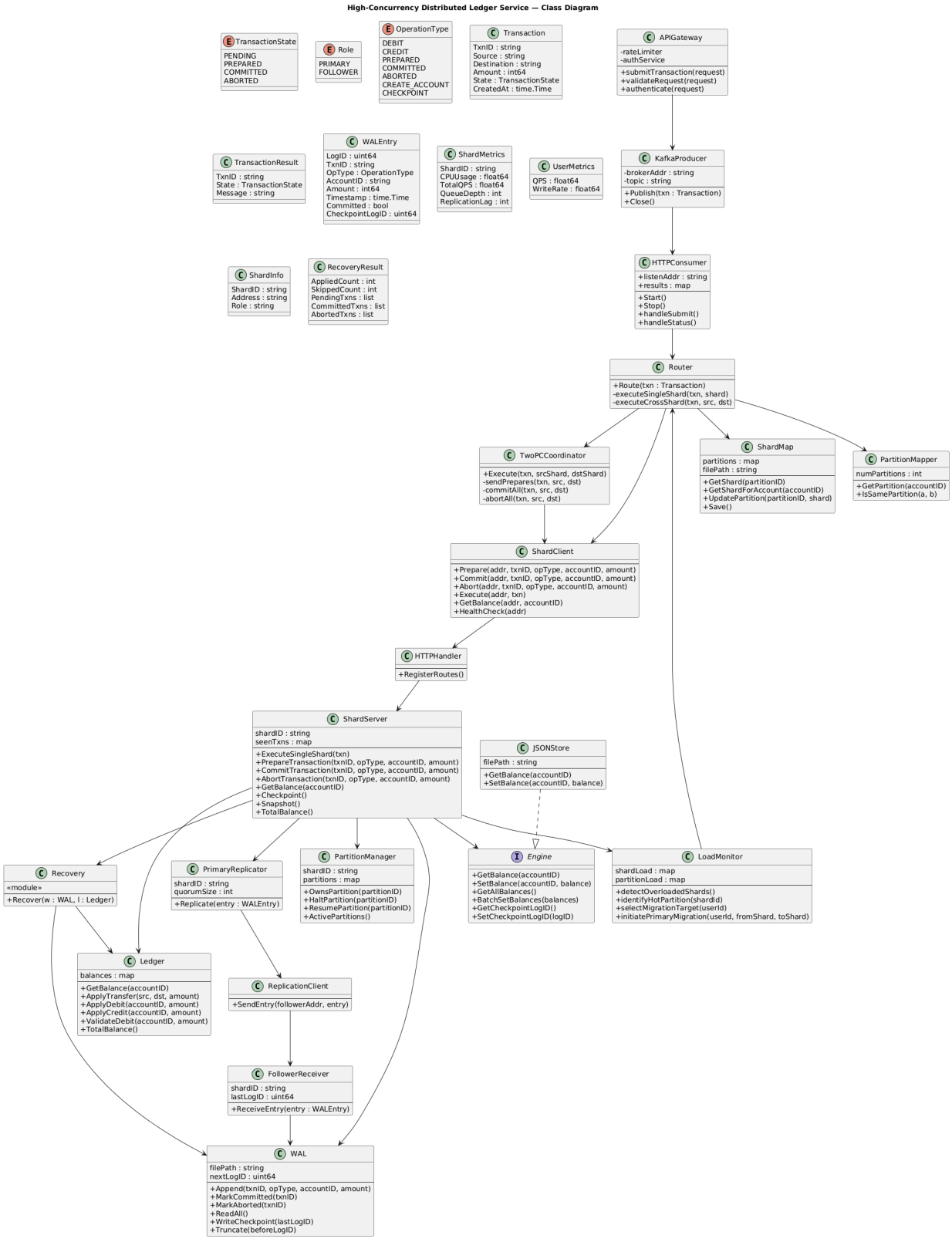

Fig. 15. Class Diagram: High-concurrency ledger service components, showing relationships between Transaction, Shard, WALEntry, Replica, and Coordinator classes.